



DiFlow

A System for Micro-Serving Text-to-Image Diffusion Workflows

Lingyun Yang*, Suyi Li*, Tianyu Feng, Xiaoxiao Jiang, Zhipeng Di, Weiyi Lu, Kan Liu, Yinghao Yu, Tao Lan, Guodong Yang, Lin Qu, Liping Zhang, Wei Wang

* Equal contribution



SHANGHAI JIAO TONG
UNIVERSITY

SOSP '26, Prague

Outline

- | | | |
|-----------|------------------------------------|--|
| 01 | Background & Motivation | Diffusion workflows, monolithic limitations, and sharing opportunities |
| 02 | DiFlow Design | Architecture, developer APIs, and GPU-native data movement |
| 03 | Scheduling | Model sharing, adaptive parallelism, and SLO-aware admission |
| 04 | Evaluation | End-to-end performance under production traffic |

Diffusion models turn noise into controllable content

A diffusion model starts from noise and repeatedly denoises a latent representation under text or image conditions.

2.5B-12B

parameters in the evaluated model families

31,133

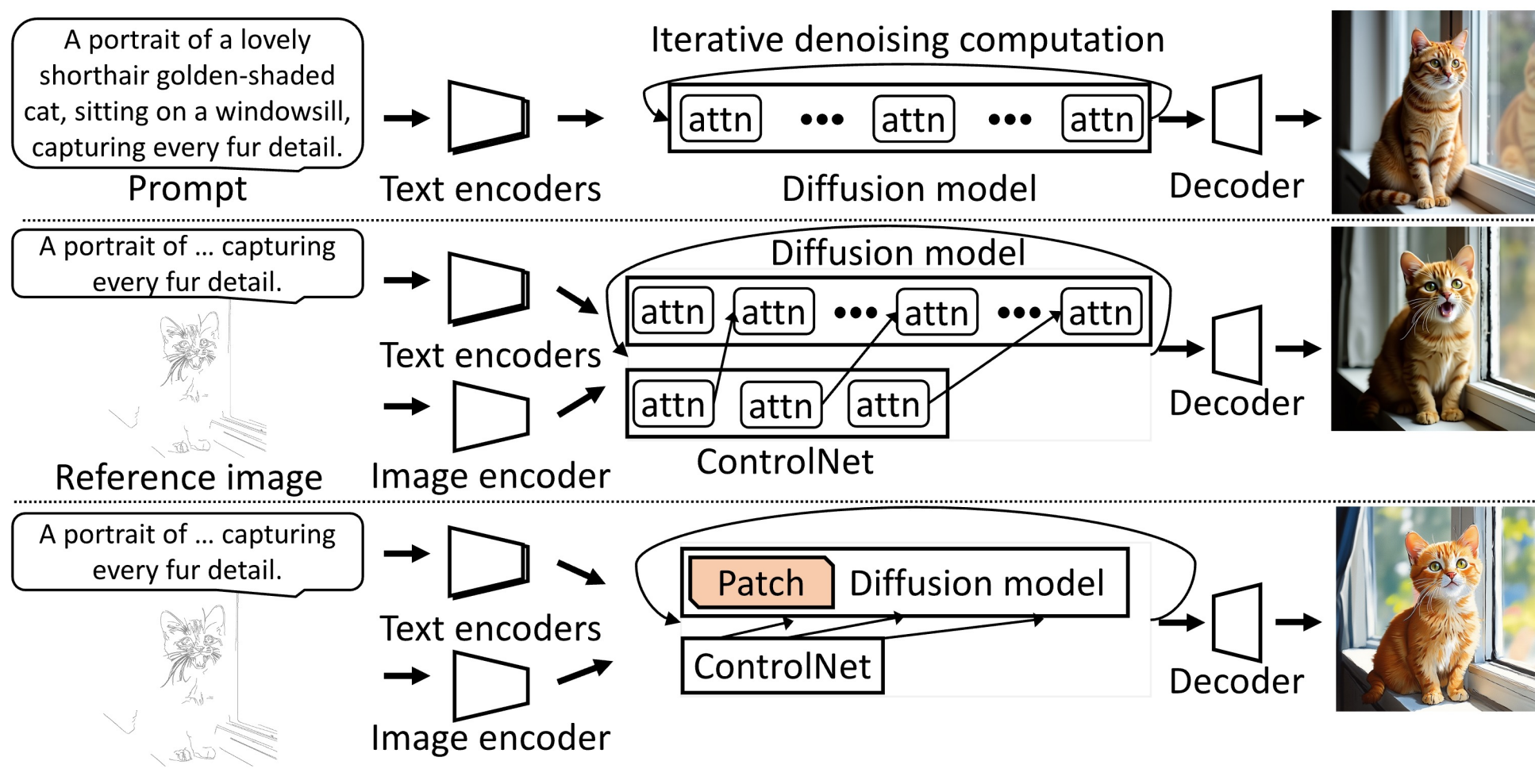
distinct workflows observed in a 20-day production trace¹



At production scale, serving becomes a workflow problem.

¹ Alibaba production trace; model scale from the DiFlow evaluation.

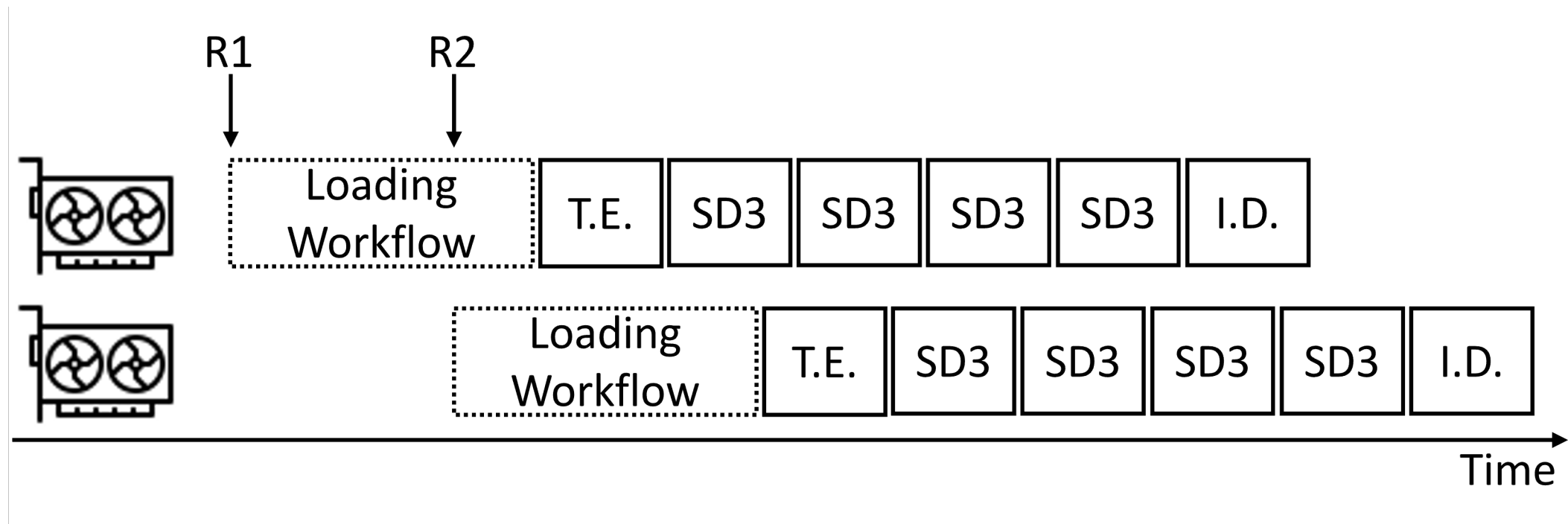
Diffusion workflows are multi-model programs



Adapters turn one pipeline into a family of workflows

Monolithic serving hides model-level structure

One workflow instance owns a fixed GPU set



A request waits for a complete workflow load, even when most models already exist elsewhere

Monolithic serving imposes four cluster-level limitations

The cluster sees one opaque workflow, so it cannot reason about individual models or their dependencies.

Limitation	What the workflow boundary hides	Cluster consequence
Full-workflow replication	Which model is saturated	Replicates 1.7-4× the base-model footprint. Adds up to 80% loading latency and wastes up to 75% GPU memory.
Replica isolation	Compatible model identity across workflows	Duplicates 2-24 GiB models and prevents cross-workflow batching on resident replicas.
Static runtime configuration	Per-model latency, throughput, and GPU needs	Leaves GPUs idle or makes a ready model wait for a fixed allocation.
Coupled workflow boundary	The component that failed or changed	Expands the failure domain and slows debugging, validation, and deployment.

Most workflow cost sits in a few components

Base diffusion model

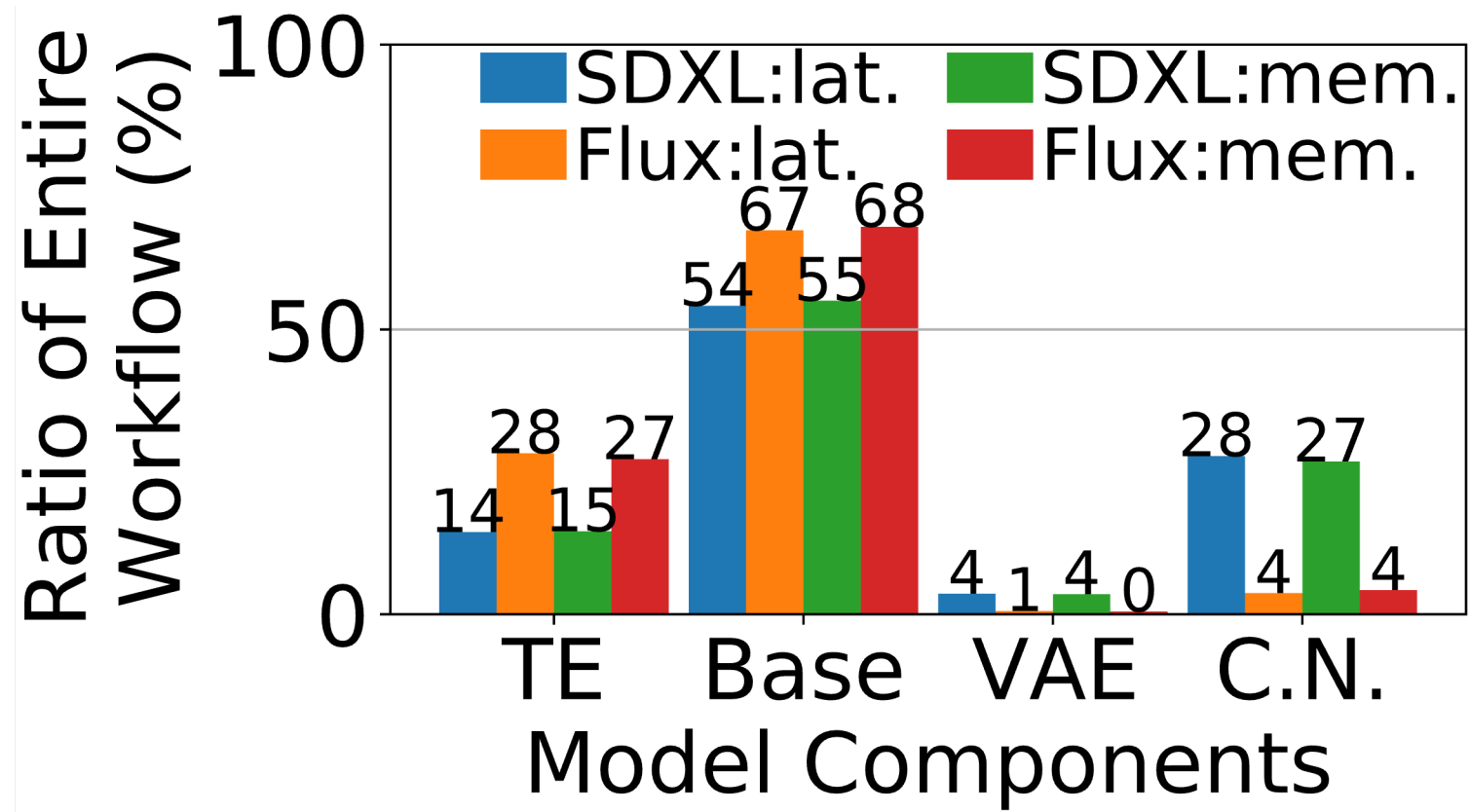
54-68%

of workflow loading time and memory

ControlNet

up to 28%

Workflow replication
duplicates every component



Shared model demand favors micro-serving

Popular diffusion backbones appear in nearly all workflows

95%

of generation requests use
one of the top five ControlNets

Micro-serving lets any compatible request reuse a resident model replica

Outline

01 Background & Motivation

Diffusion workflows, monolithic limitations, and sharing opportunities

| 02 DiFlow Design

Architecture, developer APIs, and GPU-native data movement

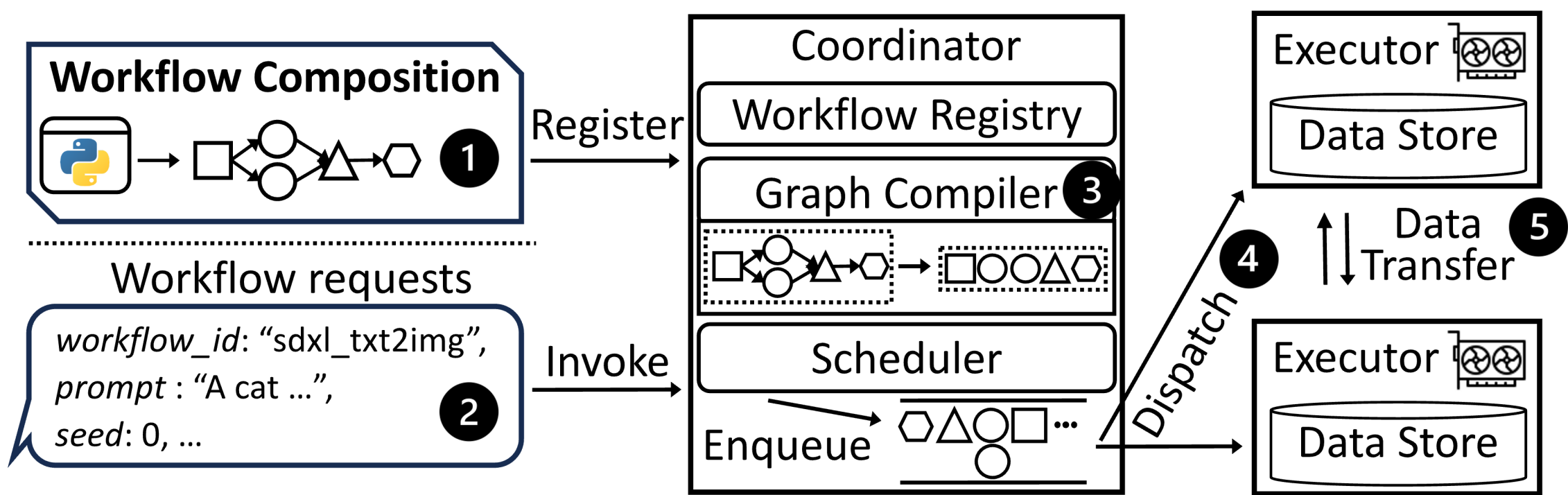
03 Scheduling

Model sharing, adaptive parallelism, and SLO-aware admission

04 Evaluation

End-to-end performance under production traffic

DiFlow decomposes workflows into schedulable nodes



Compose once in Python

Compile to model nodes

Schedule across GPU executors

A Python program becomes an execution graph

```
1 # create a workflow instance
2 workflow = Workflow(name="flux_txt2img_workflow")
3 # initialize models: LatentsGenerator, FluxTextEncoder,
4 # Flux, LoRA, and FluxVAE. All inherit the Model class
5 latents_generator = LatentsGenerator()
6 text_enc = FluxTextEncoder(model_path=model_path)
7 flux = Flux(model_path=model_path)
8 controlnet = ControlNet(model_path=controlnet_path)
9 lora = LoRA(model_path=lora_path)
10 vae = FluxVAE(model_path=model_path)
11 # initialize input placeholders for the workflow
12 seed = workflow.add_input(name="seed", data_type=int)
13 prompt = workflow.add_input(name="prompt", data_type=str)
14 num_denoising_steps = workflow.add_input(name="
    num_denoising_steps", data_type=int)
15 ref_image = workflow.add_input(name="ref_image", data_type=Image)
16 # add_patch() registers a LoRA adapter with the flux,
17 # which modifies flux model weights before inference.
18 flux.add_patch(lora)
19 # Invoke models and establish their I/O dependencies.
20 # Model invocation is expressed via __call__().
21 # For example, text_enc(prompt) means "text_enc" is
22 # invoked to process the variable "prompt".
23 latents = latents_generator(seed)
24 prompt_embeds = text_enc(prompt)
25 ref_image = vae(image=ref_image, mode="encode")
26 # Perform iterative denoising computation.
27 for i in range(num_denoising_steps):
28     controlnet_outputs = controlnet(latents, prompt_embeds, ...)
29     noise_pred = flux(latents, prompt_embeds, controlnet_outputs,
        ...)
30     # denoising the current latent to generate a new one
31     latents = denoise(noise_pred, latents)
32 output_img = vae(latents, mode="decode")
33 # initialize output placeholders
34 workflow.add_output(output_img, name="output_img")
```

Workflow developers

Use model calls, Python control flow, and adapter attachment

Model developers

Define typed inputs, loading logic, and inference logic

Graph compiler

Infers the DAG and preserves diffusion-specific operations

Developer APIs separate workflow composition from model integration

Two role-specific layers expose workflow intent and model semantics to the compiler

Workflow developers

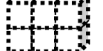
API	What it specifies
Workflow(name)	Create a workflow scope
add_input(name, data_type)	Declare an input placeholder
model(...) / __call__()	Invoke a model and record a node
add_patch() / rm_patch()	Attach or remove an adapter
add_output(value, name)	Expose a workflow result

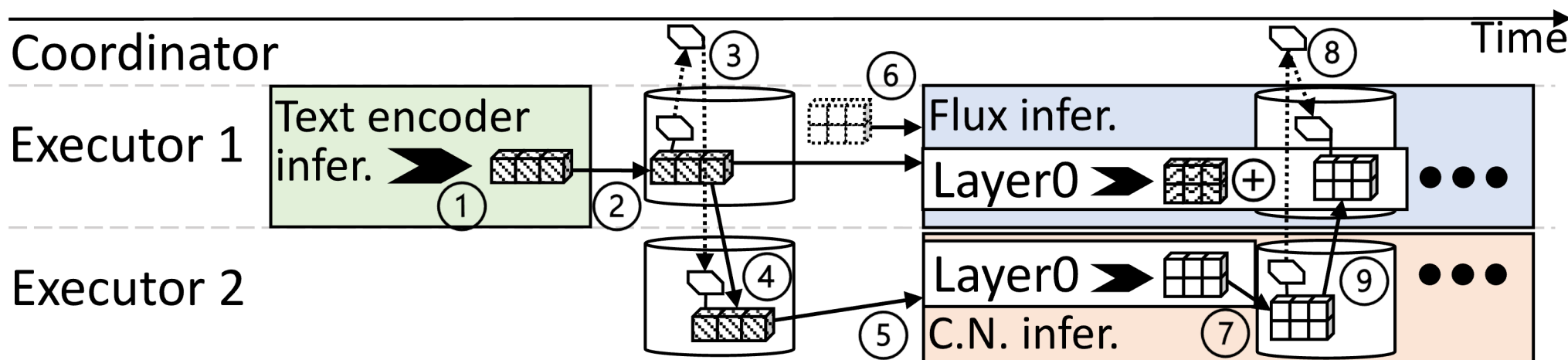
Model developers

API	What it specifies
class MyModel(Model)	Integrate a new model type
setup_io()	Declare the typed I/O contract
add_input() / add_output()	Declare outputs and eager or deferred inputs
load(model_path, device)	Initialize model components on a GPU
execute(model_components, **kwargs)	Run model inference in the backend

The compiler combines model calls with typed I/O to infer the DAG and logical data-transfer plan.

GPU-native data movement between executors

⏏ Tensor metadata    Tensors  Deferred inputs  Data store
⋯→ Metadata transfer → Tensor transfer ➡ Produce tensor ⊕ Add tensor



Eager fetch: Executer 2 fetches *prompt embedding* () from Executer 1

Deferred fetch: Executer 1 fetches *ControlNet output* () from Executer 2

Metadata goes through the coordinator

CUDA tensors move directly between executors

Outline

01 Background & Motivation

Diffusion workflows, monolithic limitations, and sharing opportunities

02 DiFlow Design

Architecture, developer APIs, and GPU-native data movement

| 03 Scheduling

Model sharing, adaptive parallelism, and SLO-aware admission

04 Evaluation

End-to-end performance under production traffic

The scheduler coordinates three runtime choices

01

Cross-workflow batching and model sharing

Compatible nodes run on any resident replica

02

Adaptive model parallelism

Idle GPUs accelerate ready model nodes

03

SLO-aware admission control

Early estimates prevent overload cascades

Model sharing preserves SLO attainment under load

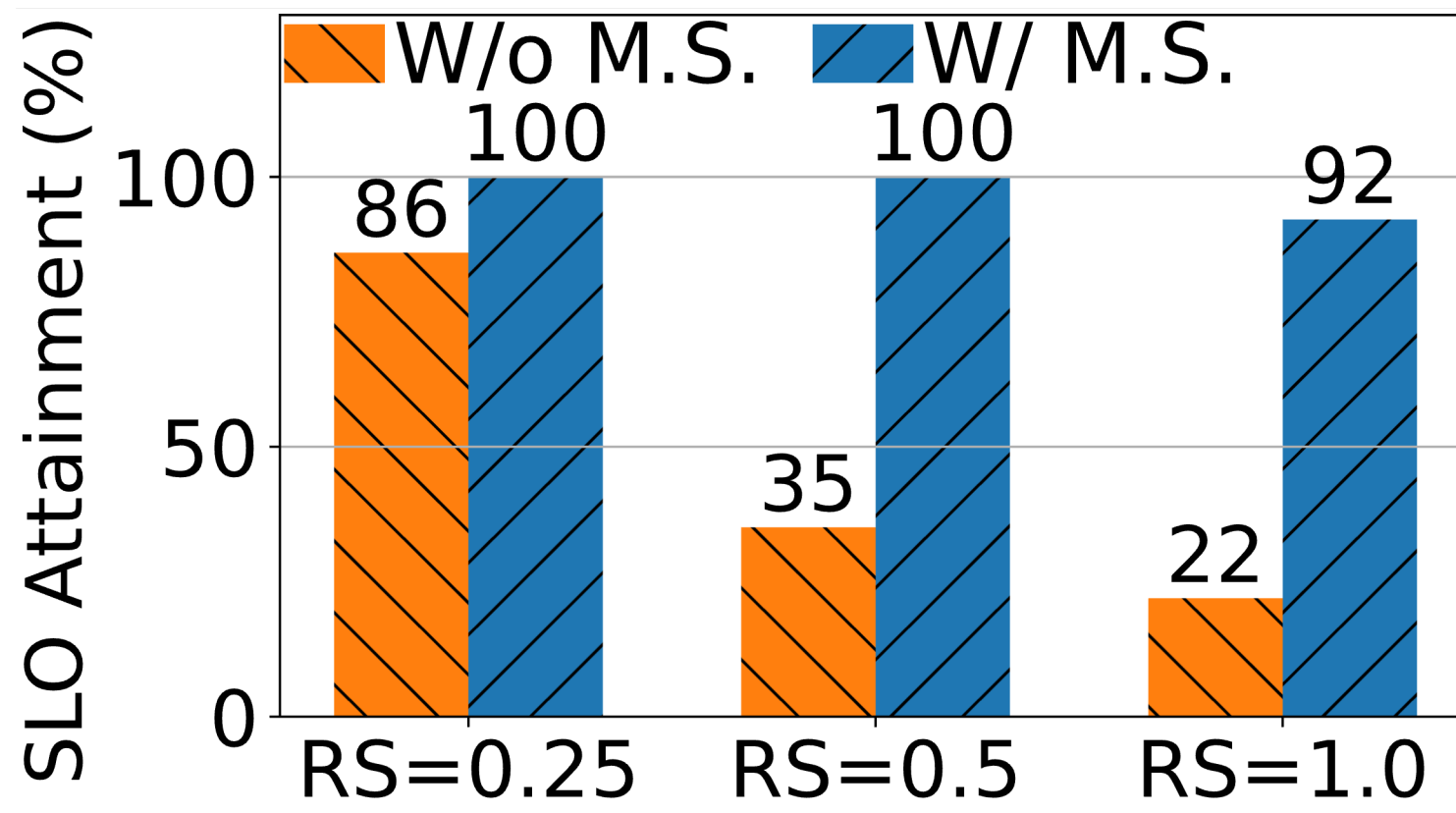
Rate scale 0.5

100% vs 35%

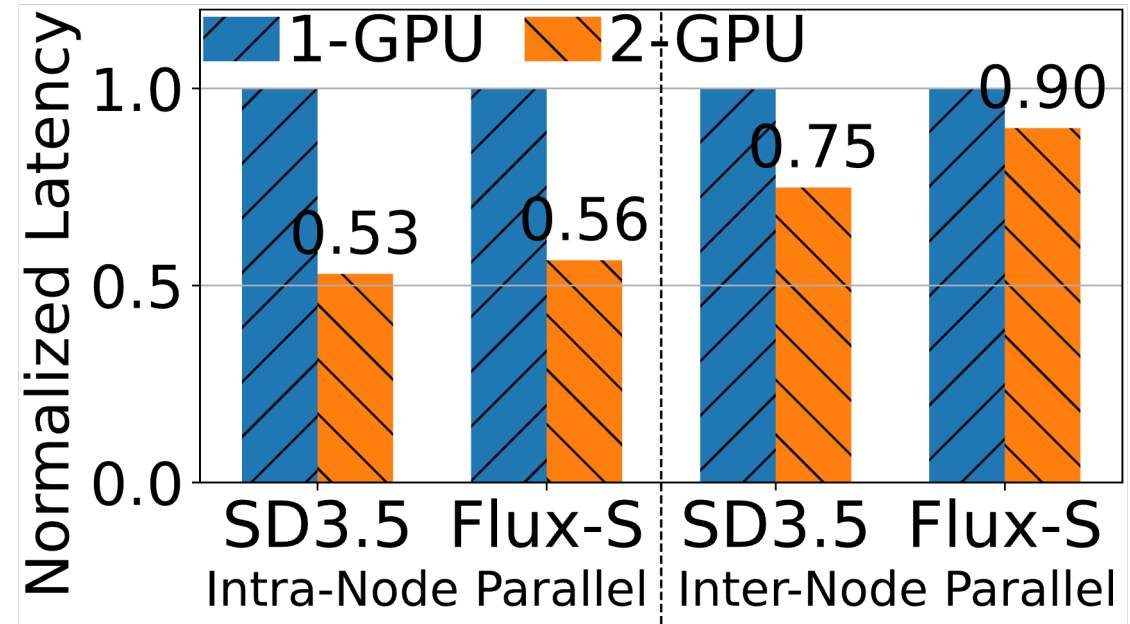
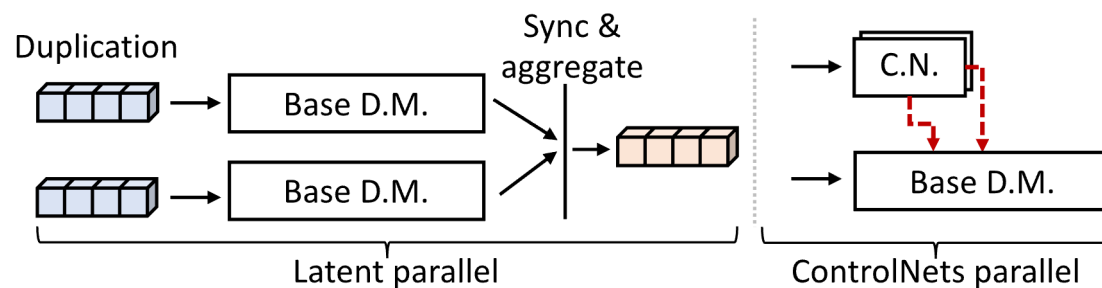
Rate scale 1.0

92% vs 22%

Sharing eliminates model switches and redundant loads



Adaptive parallelism spends idle GPUs on latency



Up to 1.9×

intra-node speedup

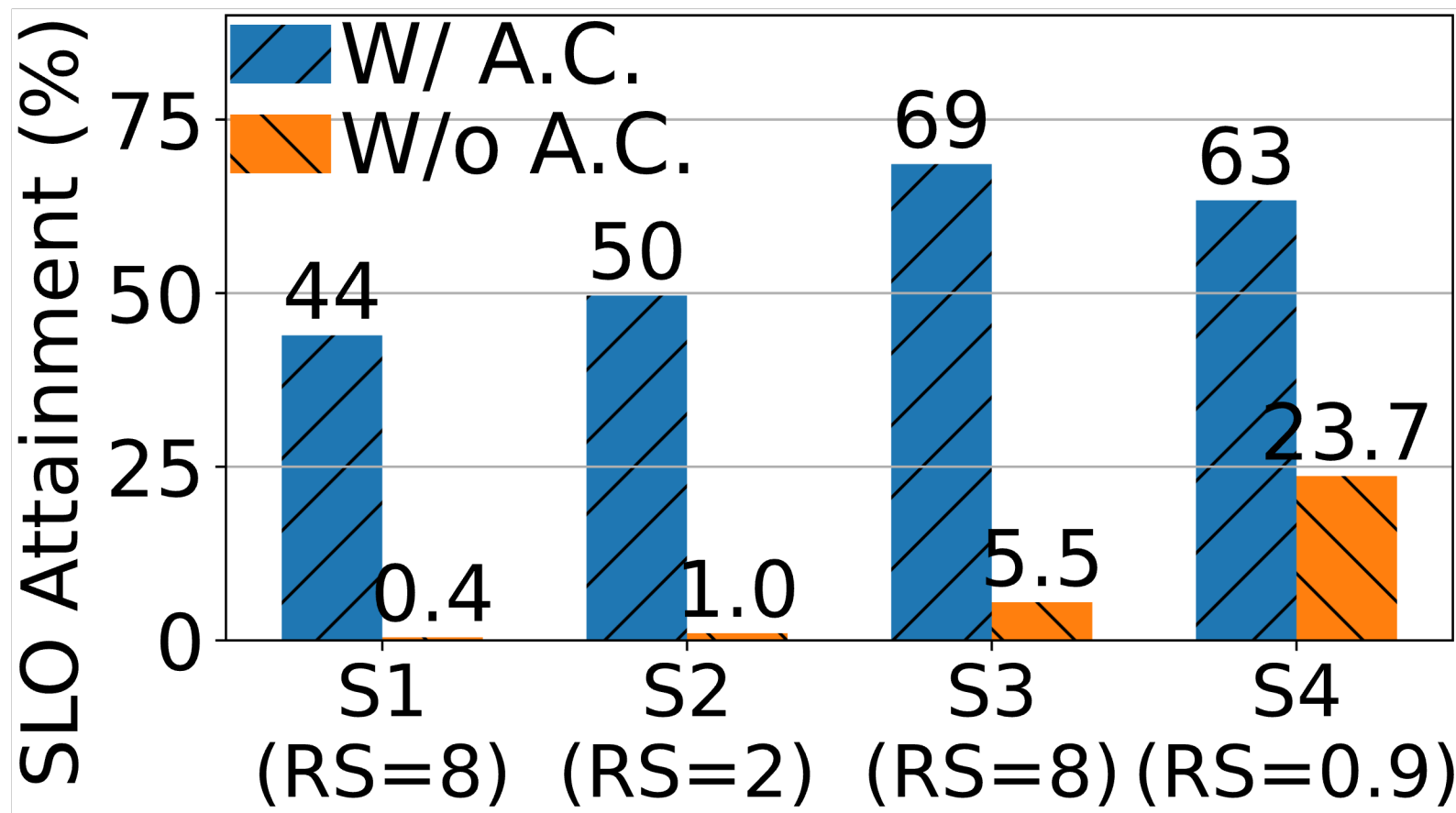
Up to 1.3×

inter-node speedup

+19 pp

SLO attainment at scale 1.0

Admission control prevents overload cascades



S1 under heavy load

0.4%

without admission control

44%

with early abort

Rejected requests remain in the SLO denominator

S1 = SD3 | S2 = SD3.5-Large | S3 = Flux-Schnell | S4 = Flux-Dev

Each setting contains one Basic workflow and two ControlNet variants.

Outline

01 Background & Motivation

Diffusion workflows, monolithic limitations, and sharing opportunities

02 DiFlow Design

Architecture, developer APIs, and GPU-native data movement

03 Scheduling

Model sharing, adaptive parallelism, and SLO-aware admission

| 04 Evaluation

End-to-end performance under production traffic

Evaluation spans six deployments and production traffic

WORKFLOWS

**SD3, SD3.5-Large,
Flux-Schnell, Flux-Dev**

DEPLOYMENTS

**Four single-model settings
Two mixed-model settings**

CLUSTER

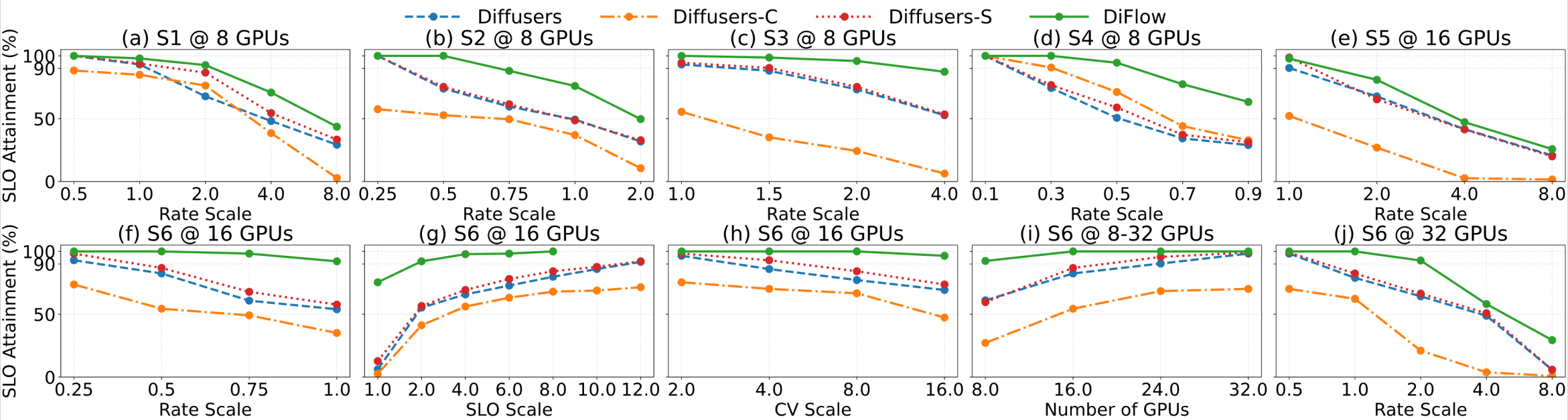
8-32 NVIDIA H800 GPUs

TRAFFIC AND METRIC

**Production arrival trace, SLO attainment
default SLO = 2× solo latency**

Baselines: Diffusers, Diffusers-C (Clockwork [OSDI'20]), and Diffusers-S (Shepherd [NSDI'23])

DiFlow sustains higher load under tight SLOs



3×

higher request rate

6×

stricter SLO

8×

higher burst CV

3×

fewer GPUs

All comparisons use a 90% SLO attainment target



Micro-serving for diffusion workflows

Model execution becomes the unit of cluster scheduling

Sharing and parallelism adapt to model demand and GPU availability

Up to 3× higher request rates and 8× higher burst tolerance



github.com/diflow-project/diflow

Questions?

